#uni

- 1 Physical Link
- 2 Data Layer (link layer)
  - 2.1.1 Framing
  - 2.1.2 Flow control
    - 2.2 Error Detection
      - 2.2.1 Parity Checking
      - 2.2.2 Internet Checksum (review)
      - 2.2.3 Cyclic Redundancy Check (CRC)
      - 2.2.4 Error Correction
        - 2.2.4.1 2-Dimension Parity checking
- 3 Reliable Data transfer (RDT) protocol
  - 3.1.1 RDT1.0: reliable transfer over a reliable channel
  - 3.1.2 RDT2.0: channel with bit errors
  - 3.1.3 RTD2.1: handling garbled ACK/NAKs
  - 3.1.3.1 Sender
  - 3.1.3.2 Receiver
  - 3.1.4 RTD2.2: a NAK-free protocol
  - 3.1.5 RTD3.0: channel with errors and loss
  - 3.1.5.1 Performance
  - 3.1.6 RTD3.1: pipelined procols operation
  - 3.1.6.1 Additional Mechanims
  - 3.1.6.2 Error recovering strategies
  - 3.1.7 Interfaces
- 4 Point To Point Protocol (PPP)
  - 4.1 Frame
  - 4.2 Byte stuffing/unstuffing
    - 4.2.1 sender (byte stuffing)
    - 4.2.2 receiver (byte unstuffing)
- 5 Multiple Access Protocols
  - 5.1 Ideal protocol
  - 5.2 Taxonomy
  - 5.3 Channel Partitioning
    - 5.3.1 Time Division Multiple Access (TDMA)
  - 5.4 Random Access
    - 5.4.1 Slotted ALOHA
    - 5.4.2 Pure ALOHA
    - 5.4.3 Carrier Sense Multiple Access (CSMA)
      - 5.4.3.1 Ethernet CSMA/CD algoritm
  - 5.5 Taking Turns MAC protocol
    - 5.5.1 Polling
    - 5.5.2 Tocken Passing
- 6 LANs
  - 6.1.1 Addressing
  - 6.1.2 Ethernet
- 7 CIAO

- 8 cane

Let's now analyze how two connected Nodes transfer data.
The simplest case is having two computers connected by the same network

1 Physical Link

Physical links are by nature unreliable and introduce errors in the communication.
The **physical layer** works as follows: bits are *encoded* by the *transmitter* through an electric/electro-magnetic/light signal and sent over the **physical link**, this signal is then *decoded* by the *receiver*.
The physical link is *unreliable*:

- signal attenuation
- noise
  - interferences, fading
- transmitter and receiver must have perfectly synchronized clocks, but also this is impossible
- it is physically impossible to produces prefect square (high voltage=1, low voltage=0) curves
  This means the **bit error rate** is never Zero: the received data sequence may be (very) different from the transmitter one.

How we achieve reliable communication over unreliable links?
We need to introduce failsafes on the **data layer**, which exists right over the physical layer.

2 Data Layer (link layer)

The data layer is implemented in a **network interface card** (NIC), or on a chip, in each-and-every host.
This NIC or chip attaches to the host's system buses.
The data link is a mix of hardware and software.
Ethernet, wifi card or chip implement both the data layer and the physical layer.

The sending side:

- encapsulates datagram in a frame
- adds error checking bits, reliable data transfer, flow control eccetera..
  The receiving side:
- looks for errors
- flow control etc
- extracts datagram and passes it to the upper layer

2.1.1 Framing

This is a service incorporated in the data link:

- we incapsulate the datagram (trama) into **frames**
- we add **header** and **trailer** to each frame

```
|    header    |     payload     |     trailer    |
```

Obviously we introduce overhead.

2.1.2 Flow control

This is another possible service on the data link: receiver decides pacing between transmissions.

2.2 Error Detection

The receiver finds corrupted bits, then it can recover automatically the bits or ask the transmitter to re-send the corrupted frames.

- **Half-Duplex**: nodes at both ends of link can transmit, but not at the same time: achieved over a physical link by putting at both end **transceiver**, which is both a transmitter and a receiver.
- **Full-Duplex**: we put two lines of physical link, one is used to transmit by one end, the other line is used to transmit by the other end.

The message includes:

- **R**: error detection bits (redundancy bits), these are included in the **trailer** (because these are calculated while transmitting the message)
- **D**: data protected by error checking, may include header fields

What we do is apply a certain *mathematical operation* to the data, and include the result in the encoded message.
When the receiver gets the message, it applies the same operation to the D data bits, and then checks if the resulting redundancy bits are the same as the received redundancy bits.

So D and R are the sent bits and D' and R' are the received bits. If the result of the operation on D' is different from the received R' bits, the message changed and we have N errors. This could mean the D' bits are wrong, or the R' bits are wrong.

Here are some of the commonly used operations:

2.2.1 Parity Checking

This method detects single bit errors.
Only one bit, the **parity bit**, is added as trailer to the D bits.

- Even parity: set parity bit so there is an even number of 1's
- Odd parity: set parity bit to there is an uneven number of 1's

Used between computers because sent messages usually only go up to 64bits

2.2.2 Internet Checksum (review)

The sender:

- treats the package as a sequence of 16-bit integers
- checksum: the sum of the integers
- checksum value put into the **checksum field**
  The receiver:
- calculates the checksum and compares it to the received one in the checksum field

- if these are not equal an error is detected

Used in the TCP protocol, assuming the lower layers have already checked.

2.2.3 Cyclic Redundancy Check (CRC)

This is a more powerful error detecting coding.

- **D**: data bits
- **G**: bit pattern (generator), of r+1 bits (known by both sender and receiver)
  Sender:
- calculates the trailing R bits

```
bit pattern:
   |       D        |   R    |

Formula per bit pattern:
<D,R> = D*2^r XOR R
```

The goal is to choose r CRC bits and R, such that <D,R> is exactly divisible by G (mod 2)
Receiver:

- receiver knows G and divided <D,R> by it. If the remainder of the result is not zero and error is detected

This operation can detect all burst errors that are less than r+1 bits

Widely used in practice, in Ethernet, 802.11 Wifi and more.
Used in the data link.

2.2.4 Error Correction

We now want to not only detect that an error occurred, but also want to localize the wrong bits.

- **EDC**: error detection and correction bits
- **D**: data

2.2.4.1 2-Dimension Parity checking

Works the same as Parity Checking but we divide the message in a matrix where every line and also every column has its parity bit. This helps us localize the wrong bits.

3 Reliable Data transfer (RDT) protocol

Sender utilizes rdt_send(), receiver uses rdt_receive.
We will incrementally develop the sender and the receiver sides of the reliable data transfer (RDT) protocol.
We will use finite state machines (FSM) to descrive the sender and the receiver.

3.1.1 RDT1.0: reliable transfer over a reliable channel

The underlying channel is supposed to be perfectly reliable

- no bit errors

- no loss of packets
  There are *separate* FSMs for the sender and the receiver
- sender send data into underlying channel
- receiver reads data from underlying channel

3.1.2 RDT2.0: channel with bit errors

Now the underlying channel may flip bits in packet
Question: how to recover from errors?

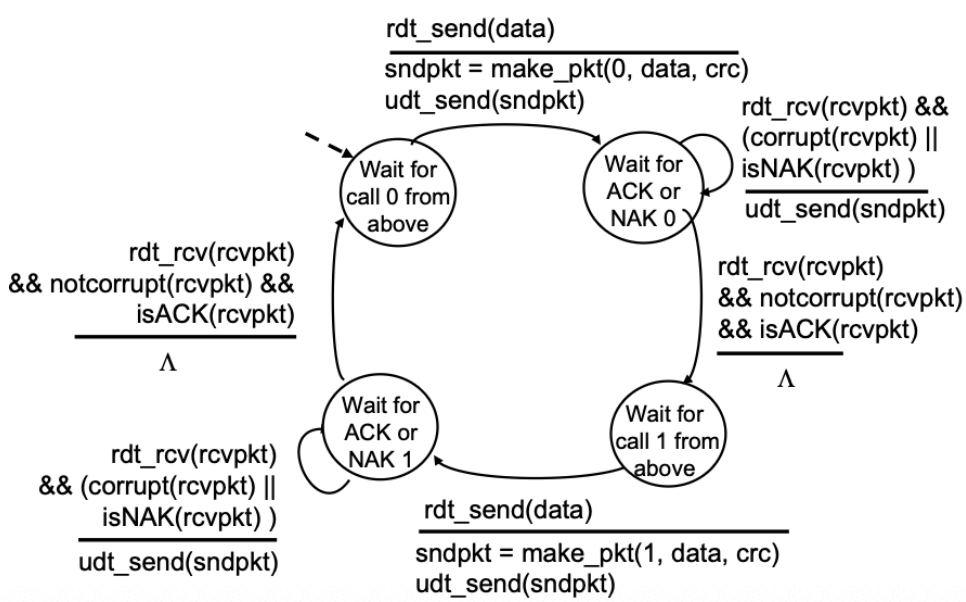
- **Acknowledgements** (ACKs): receiver explicitly tell sender that the packet recieved is OK
- **Negative acknoledgements** (NAKs): receiver explicitly tell sender that the packet recieved has errors
  - The sender *re-transmit* the packet
  The used technique is **stop and wait**. The sender sends the packet then it stops and wait for the reponse from the receiver.

3.1.3 RTD2.1: handling garbled ACK/NAKs

What if the acks and naks get currupted?
Now the sender do not know is the receiver got the right packet or not and it can't just retransmit causing possible duplicates.
A *solution* is for the sender to add a **sequence number** to each packet so the receiver can discard duplicates.

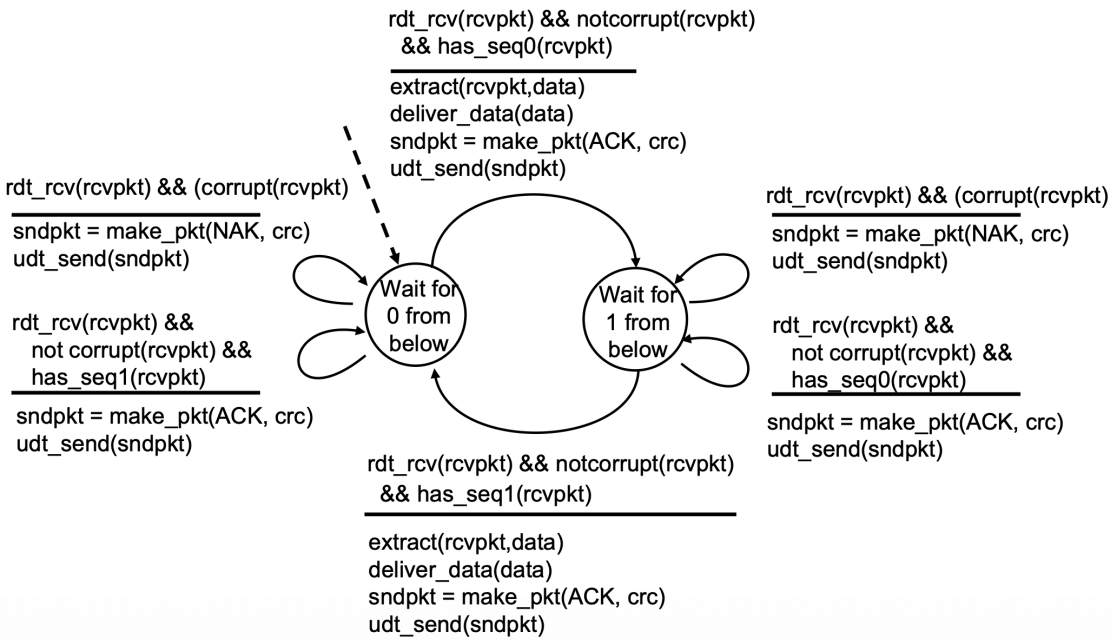
3.1.3.1 Sender

- Adds a sequence to the packet
- Must check if the recieved ACK/NAK is corrupted
- Twice as many states
  - state must "remember" whether the "expected" packet should have a sequence of 0 or 1



3.1.3.2 Receiver

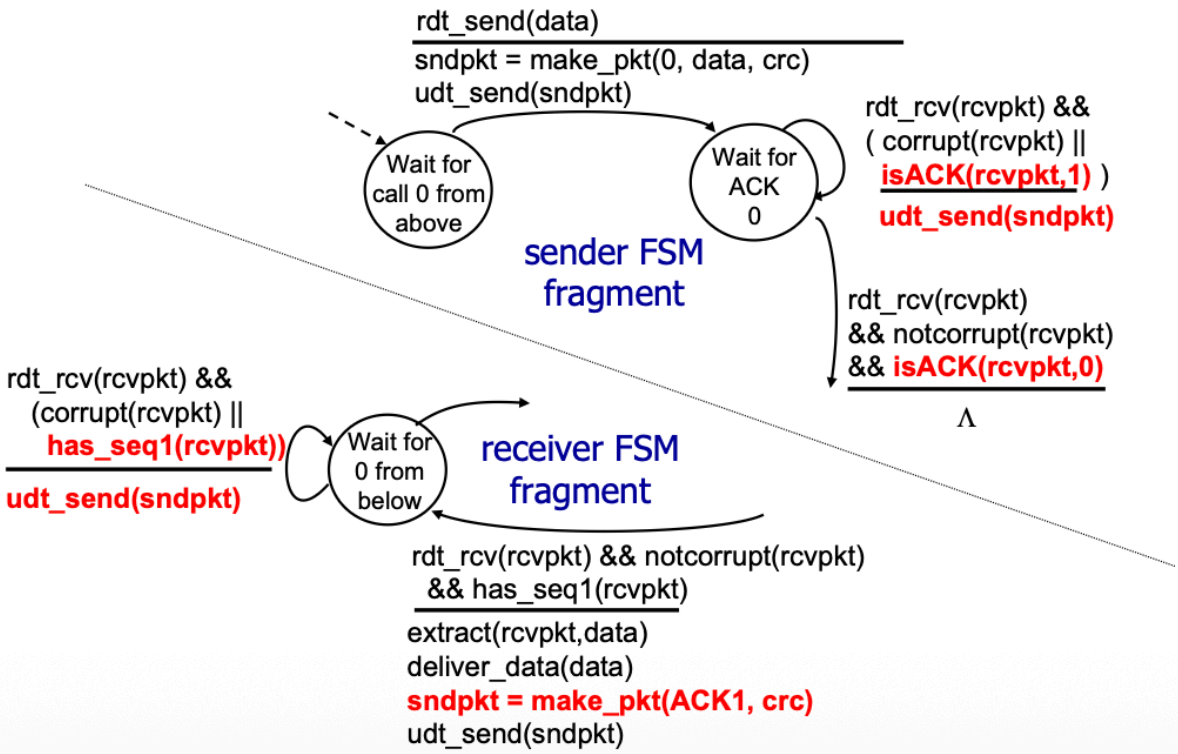
- Must check if the recieved packet is duplicate
  - state indicates whether 0 or 1 is the expected packet sequence
- receiver can't know if the last ACK/NAK was recieved OK at sender



3.1.4 RTD2.2: a NAK-free protocol

It as the same functionality as the 2.1 but without NAKs.
Instead of a NAK the receiver sends ACKs for the last packet which was recieved OK.

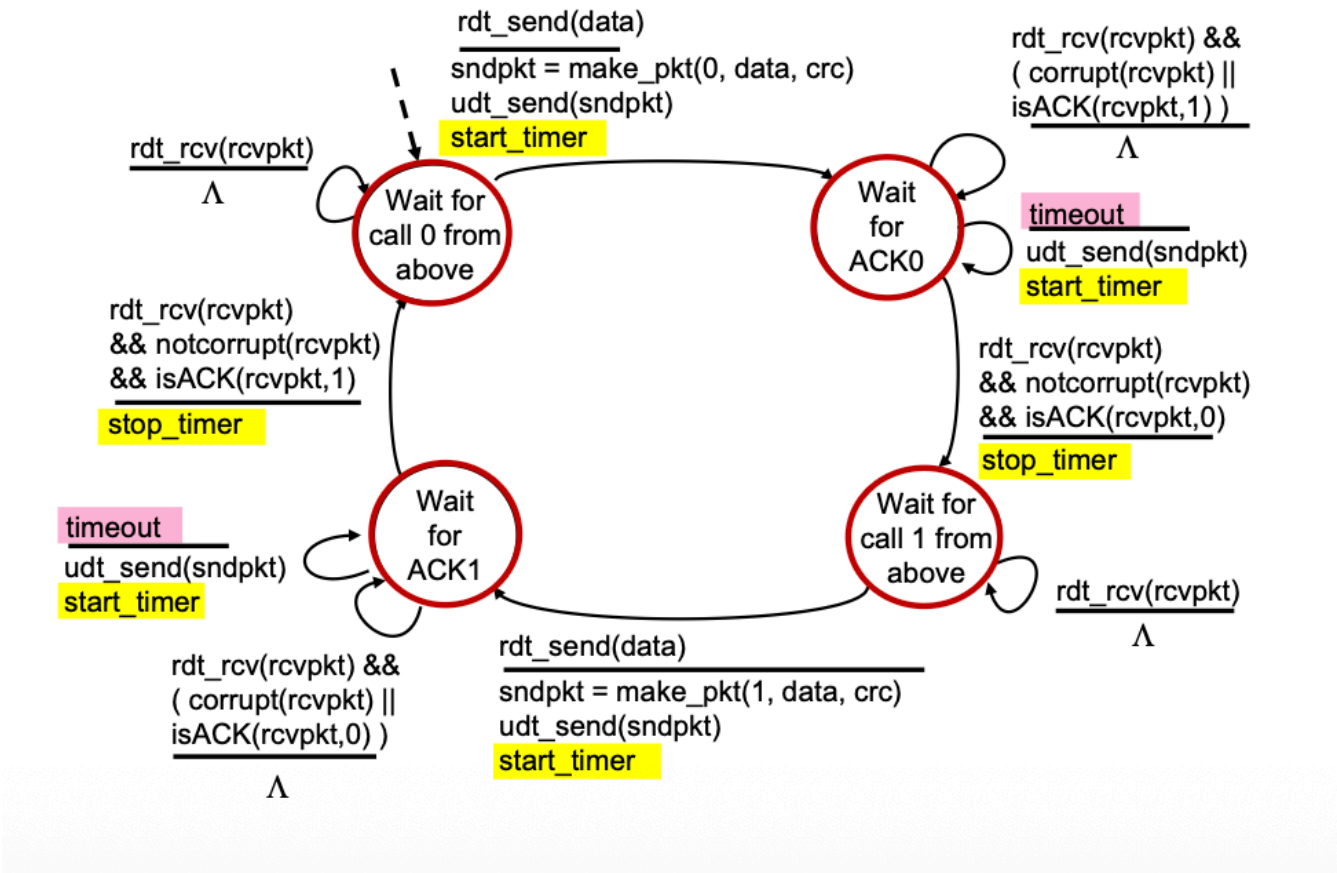
- the receiver must explicitly include a sequence of the packet which is being ACKed
  duplicate ACKs result in a single NAK so the sender retransmit.
  Many protocols including TCP use this approach to be NAK-free.



3.1.5 RTD3.0: channel with errors and loss

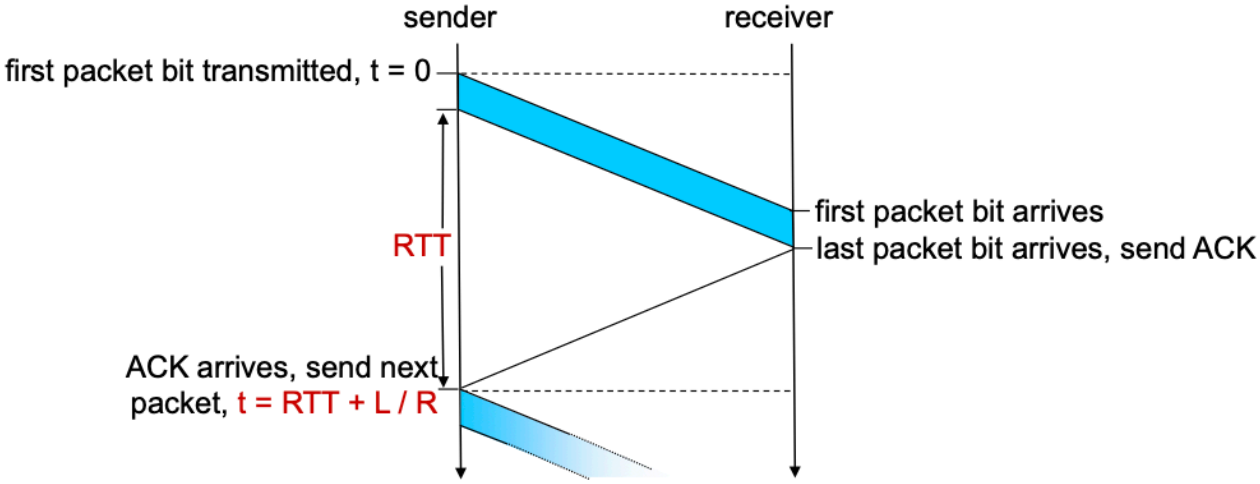
New channel assumptions -> the underlying channel can also lose packets (data, ACKs).

- checksum, sequence, ACKs, retransmission will be of help but not quite enough
  The used approach:
  - The sender waits a "reasonable" amount of time for ACK
  - Retransmit if the ACKs is not recieved in time
  - If the packet is just delayed
  - retransmittion will send duplicate but the seq already handles this
  - the reciever must specify a sequence to the ACKed packets



3.1.5.1 Performance

Define U*sender the **utilization** which is the fraction of time the sender is busy sending.*
Let's say we have a 1Gbps link, 15 ms prob. delay, 8000 bits packet.
The time to transmit packet into the channel is $D*{trans} = \frac{L}{R} = \frac{8000 \text{bits}}{10^9 \text{bits/sec}} = 8 \text{ms}$



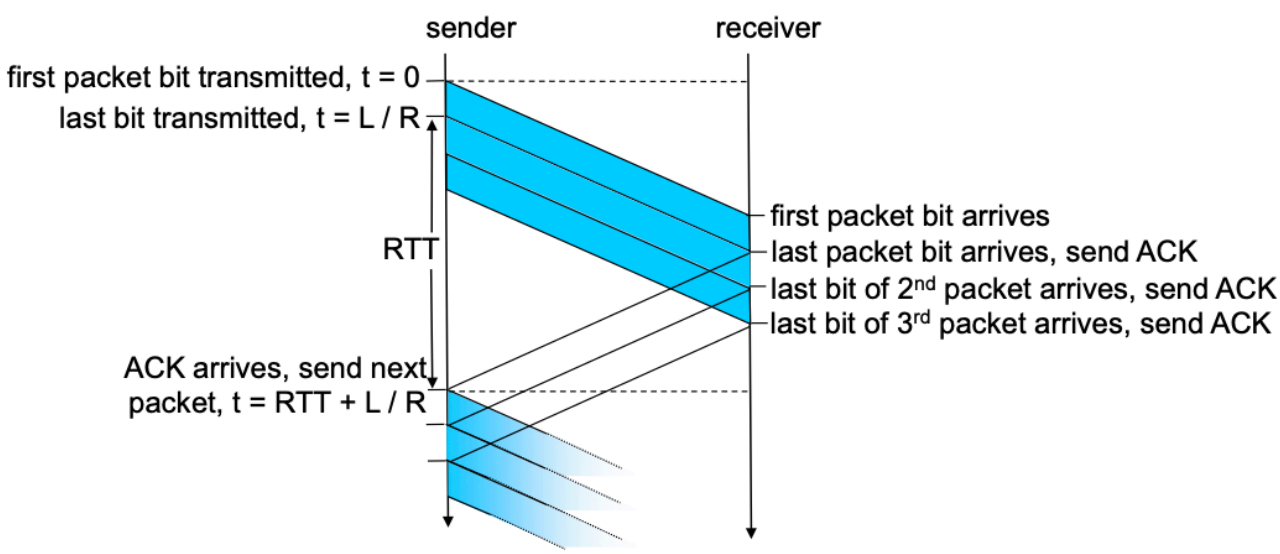
so

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{D_{trans}}{RTT + D + trans} = \frac{0.008}{30.008} = 0.00027$$

The RDT protocol performance stinks.

3.1.6 RTD3.1: pipelined procols operation

Pipelining: sender allows multiple, "in-flight", *yet-to-be-acknowledged* packets.



so

$$U_{sender} = 3\frac{D_{trans}}{RTT + D_{trans}} = 0.00081$$

3-packet pipilining increases utilization by a factor of 3!

3.1.6.1 Additional Mechanims

- range of sequence numbers must be **increased**
- **buffering** at sender and/or receiver
  - the sender must buffer all packets not yet acknowledged
  - the receiver may buffer *out-of-order* packets

- the range of sequence numbers and buffer size depend on how the protocol manages **lost, corrupted and delayed packets**

3.1.6.2 Error recovering strategies

3.1.6.2.1 Go-Back-N

- *sender*: up to N unACKed packets in pipeline
- *reciever*: only sends cumulative ACKs
- *sender*: has time for oldest unACKed packet
  - if time expires -> retransmit all unACKed packets

3.1.6.2.1.1 Sender

the sender window of up to N, consecutive tansmitted but unACKed packets.
A k-bit sequence number is included in the packet header.

3.1.6.2.1.2 Reciever

It only sends AKCs for correctly received **highest** in sequence order packets.

- it may generate duplicate ACKs
- it only need to remeber the `rcv_base`
  If it receive out of order packet
- can discard or buffer: implementation decision
- re-ACK packet with the highest in order sequence
  Selective Repeat
- *sender*: up to N unACKed packets in pipeline
- *reciever*: ACKs individual packets
- *sender*: maintains time for each unACKed packet
  - if timer expires -> retransmit only unACKed packets

3.1.7 Interfaces

4 Point To Point Protocol (PPP)

This is a protocol at the data link level (data layer).
This protocol was defined in a period in which the TCP/IP stacks was only one of the possible internet architectures.
Features:

- packet framing
- error detection (Cyclic Redundancy Check (CRC) algorithm)
- no error correction! no retransmission! -> *unreliable service*
- **bit transparency**: no limitations on what bit patterns can be transmitted
- network layer address negotiation
- no point-to-multipoint communication
- no flow control

4.1 Frame

```
| flag | address | control | protocol | info | check | flag |
   1B       1B        1B       1,2B     varL    2,4B     1B
```

- flags are delimiters for the framing, and are set to `01111110`
- address is useless, and always set to $0xFF$. Historically address with every bit set means Broadcast.
- control does nothing, could be used in the future
- protocol specifies to which upper layer protocol to deliver the frame (e.g. IP)
- info is the data carried for the upper layer, and is so obviously of variable lenght
- check is for the [Cyclic Redundancy Check (CRC)]() for error detection

4.2 Byte stuffing/unstuffing

How do we ensure bit transparency?
How do we make sure that info in not confused with the escape flag?
We can use and escape sequence: to send 01111110 the sender sends 01111110 01111101

4.2.1 sender (byte stuffing)

```
01111110 -> 01111101 01111110
01111101 -> 01111101 01111101
```

4.2.2 receiver (byte unstuffing)

```
01111101 01111110 -> 01111110
01111101 01111101 -> 01111101
```

5 Multiple Access Protocols

There are two types of links

- **point-to-point**
  - PPP link between Ethernet switch, host
  - PPP for dial-up access
- **broadcast** (shared wire or medium)
  - old-fashioned Ethernet
  - upstream HFC in cable-based access network
  - 802.11 wireless LAN, 4G/4G. satellite

  Two or more simultaneous trasmissions on the same broadcast channel *interfere*. This lead to **collision**.

We need a **multple access procotol** since communication about the channel sharing must use the channel itself.
It's an algorithm that deremines how the nodes share the channel AKA it determines which nodes can transmit

5.1 Ideal protocol

Suppos we have a **MAC** (multiple access protocol) or rate R bps.

1. when one nodes wants to transmit, it can send at rate R.

2. when M nodes wants to transmit, each can send at average rate R/M.
3. fully decentralized:
    - no special node to coordinate transmissions
    - no synchronization of clocks or slots
4. KISS (Keep It Stupid Simple)

5.2 Taxonomy

There are three borad classes:

1. **Channel partitionnig**
    - divide channel into smaller "pieces" (time slots, frequency, code)
    - allocate piece to node for exclusive use
2. **Random access**
    - channel not divided, allow collisions
    - "recover" from collisions
3. **Taking turns**
    - nodes take turns, but nodes with more to send can take longer turns

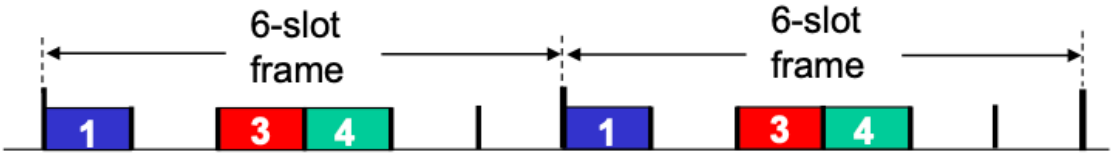
5.3 Channel Partitioning

5.3.1 Time Division Multiple Access (TDMA)

It's a channel parititionig protocol.
The access to channel is divided in "rounds" where each stations gets a fixed lenght slot.

6-station LAN:

- 1,3,4 have packets to send.
- slots 2,5,6 are idle.



5.4 Random Access

When a node has a packet to send it transmit it at full channel data rate R.
If a collision occurr it needs to be deceted and handled.
The protocol therefore needs to specify how to detect collisions and how to recover from them.

5.4.1 Slotted ALOHA

assumptions:

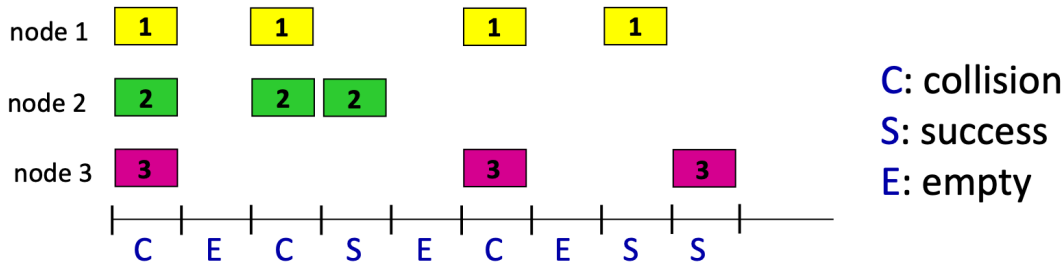
- all frame are the same size
- time is divided into equal size slots
- nodes start to transmit only at the beginnig of a slot

- nodes are syncronized
- if two or more nodes tansmit in the same slot all the nodes detect the collision

  Operations:
- when a node obtains fresh frame it transmit in the next slot

  - *no collision*: a node can send a new frame in the next slot

  - *collission*: the node retransmit the frame in each subsequent slot with probability p until it succeed



  Pros:
- a single active node can continuosly transmit at full rate of channel
- highly decentrelized: only slots in nodes need to be in sync
- simple

  Cons:
- collision -> waisting nodes
- idle slot
- nodes may be able to detect collision in less than time to transmit packet
- clock synchtonization

How to make it effcient?

suppose: N nodes with many frames to send, each transmit in slot with probability p

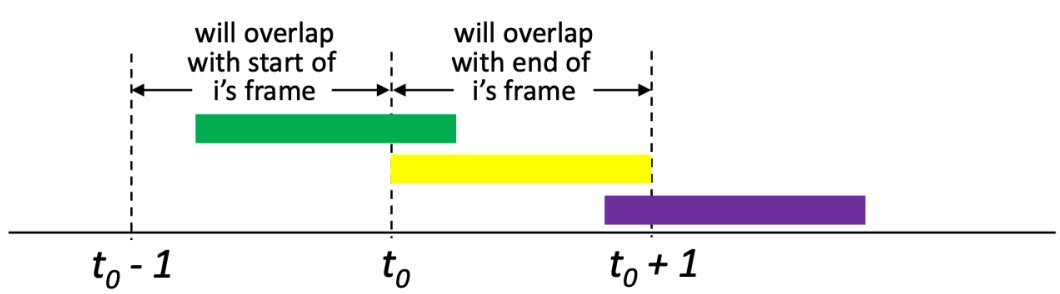
- probability that a given node has success in a lost is $p(1-p)^{N-1}$
- probability that any node has a success is $Np(1-p)^{N-1}$
- to find $\overline{p}$ that maximize $Np(1-p)^{N-1}$ -> $\lim_{N\to\infty} N\overline{p}(1-\overline{p})^{N-1}$
- max efficiency is $\overline{p} = \frac{1}{e} = 0.37$

5.4.2 Pure ALOHA

also called *unslotted*, simplet with no syncronization.

- when a node obtains a fresh frame it transmit immediately.
- no syncronization mean higher chance of collisions.

  - frame sent at t_0 collides with other frames sent in $[t_0 - 1, t_0 + 1]$



  Pure aloha efficency is 0.18!

5.4.3 Carrier Sense Multiple Access (CSMA)

simple **CMSA**
It's based on the concept: **lister before transmit**

- if the channel is sensed *idle*: transmit the entire frame
- if the channel is sensed *busy*: defer transmission
  human analogy: do not interrupt others

CSMA with collisions: **CSMA/CD**

- collisions are detected within short time
- colliding transmission are **aborted**, reducing channel **wastage**
- collision detection is easy in a wired enviroment, diffucult in a wireless one

Collisions can still occur with carrier sensing

- **propagation delay** means two nodes may not hear each other's just started transmission

5.4.3.1 Ethernet CSMA/CD algoritm

1. NIC receives datagram from network layer, creates frame
2. NIC senses channel:
   - *idle*: start frame transmission
   - *busy*: wait until channel idle, then transmit
3. if NIC transmits entire frame without collision, NIC is done with the frame!
4. If NIC detects another transmission while sending: abort, send jam signal
5. After aborting, NIC enters **binary (exponential) backoff**
   - after mth collision, NIC chooses K at random from $\{0, 1, 2, \ldots, 2^{m-1}\}$ then waits $K * 512$ bit times, returns to step 2

the collission are detected comparing strength of the output signal with the strength of the link signal.

5.5 Taking Turns MAC protocol

it tries to get the best of [5.3 Channel Partitioning]() and [5.4 Random Access]()
and combine it.

5.5.1 Polling

A master node invites other nodes to transmit in turn.
Thhis is tipically use with "dumb" devices.

Concerns:

- polling overhead
- latency
- single point of failure (master)

5.5.2 Tocken Passing

The control **token** is passed from one node to the next sequentially.
This resolve a few issues with [5.3.1 Time Division Multiple Access (TDMA)](): no idle slots

are wasted for example.

Concerns:

- Tocken overhead
- latency
- single point of failure (token)
  - When the tocken fails, the node tasked with emitting the token in the network at some point recognize the failure and emits another token in the network

6 LANs

6.1.1 Addressing

6.1.2 Ethernet